

UNIVERSIDADE FEDERAL DE MINAS GERAIS
DEPARTAMENTO DE CIÊNCIA DA COMPUTAÇÃO
ALGORITMOS E ESTRUTURAS DE DADOS III

DICKSON ALVES DE SOUZA

IMPLEMENTAÇÃO DA BIBLIOTECA BIGNUM USANDO A LINGUAGEM C

BELO HORIZONTE
29 DE AGOSTO DE 2011

1. INTRODUÇÃO

O tratamento de números inteiros é dependente da linguagem de programação e da arquitetura do computador. Algumas linguagens, como Java, C# e Python [1], possuem suporte ao uso de grandes números inteiros, seja como recurso da própria linguagem ou através de implementação de bibliotecas padrão da linguagem. Outras linguagens, como por exemplo, C e C++, não possuem acesso nativo a essa funcionalidade, porém permitem a implementação usando o conceito de tipo abstrato de dados.

A manipulação de números inteiros com precisão arbitrária, como são conhecidos, é necessária em campos como criptografia de chave pública usando algoritmos RSA, onde, no mínimo, 100 dígitos são exigidos para garantir segurança, mas é usual utilizar 1000, e no estudo de conjecturas sobre números [2].

Nesse trabalho, implementou-se uma biblioteca para manipulação de inteiros de precisão arbitrária, através do TAD bignum.

2. OBJETIVOS

O objetivo desse trabalho foi propiciar a familiarização com recursos da linguagem C através da implementação de tipos abstratos de dados (TAD's) para manipulação de inteiros de precisão arbitrária, utilizando-se de alocação dinâmica de memória.

3. METODOLOGIA

O TAD bignum foi projetado para conter uma lista encadeada de dígitos de um número de precisão arbitrária. Para manipular o sinal do número, incluiu-se uma variável como visto no struct abaixo:

```

struct bignum
{
    LIST* list_num;
    char sign;
}; // struct bignum

```

A lista encadeada, por sua vez, operando com nós denominados dígitos, por analogia com o problema em estudo, forneceu todas as funções essenciais para as diversas operações pelo TAD bignum. Optou-se por construir os nós da lista com três campos, um campo para armazenar um ponteiro para um nó anterior na lista, um segundo campo para o valor do dígito e o último campo para armazenar um ponteiro para o nó seguinte. Essa decisão foi tomada em virtude de especificações de dois tipos de funções do TAD bignum: as funções para impressão necessitam, para maior eficiência, ler o número a partir de seu dígito mais a esquerda até alcançar o último dígito. Por outro lado, as operações matemáticas, soma, subtração e multiplicação, são compreendidas e implementadas acessando os dígitos da direita para a esquerda. Assim, embora considerando o overhead de memória causado por um ponteiro adicional, adotou-se tal estratégia na implementação da lista encadeada.

O struct list é apresentado abaixo para visualização de seu modelo de implementação:

```

struct list
{
    DIGIT* first_;
    DIGIT* last_;
    unsigned long int size_;
}; //struct list

```

Nota-se a presença de dois ponteiros, um para o primeiro dígito e outro para o último dígito, para propiciar a versatilidade da execução de dois tipos de funções como comentado anteriormente e uma variável do tipo unsigned long int, projetada para armazenar o tamanho da lista, evitando percorrer a lista completa, como seria na situação de ausência de uma variável para armazenar o tamanho atual da lista. A escolha do tipo dessa variável foi baseada em admitir que o teto para essa variável seria adequado para maior parte das aplicações dessa biblioteca (cerca de 4 bilhões de dígitos), lembrando que antes de atingir essa monta no tamanho da lista, provavelmente o limite de memória física disponível pode ter sido atingido.

Para encerrar a discussão sobre os tipos criados, temos o TAD digit, constituído de três variáveis ou membros: dois ponteiros e uma variável do tipo char. Um ponteiro indica a posição de memória do dígito anterior e o outro indica a posição de memória do dígito seguinte.

```
struct digit
{
    char dig_;
    DIGIT* previous_;
    DIGIT* next_;
}; //struct digit.
```

Cada dígito representa 12 bytes ocupados na memória do computador, na arquitetura empregada na compilação desse trabalho (PC x86 32 bits, Ubuntu 11.04). O acréscimo em relação à estrutura de uma lista encadeada simples (com apenas um ponteiro em cada dígito) é de 4 bytes.

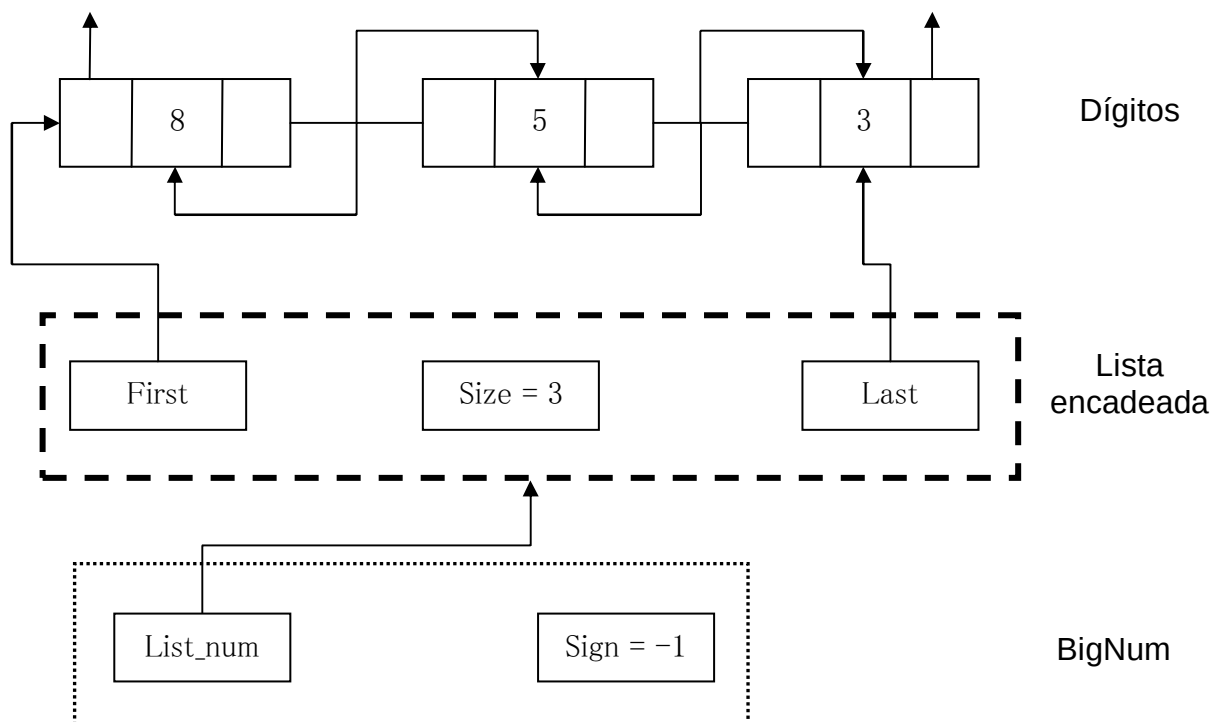


Figura 1: Representação do número -853 no modelo proposto nesse trabalho

4. ESTRUTURA DOS MÓDULOS

Cada um dos três TAD's implementados constitui-se de um arquivo cabeçalho (nome do tad.h) contendo os protótipos para as funções, seus pré-requisitos e formato de saída e um arquivo fonte (nome do tad.c), contendo as definições das funções. Basicamente, buscou-se implementar o mínimo de funções necessárias à manipulação plena de cada TAD, seguindo o paradigma de fornecer funções construtoras (não recebem argumento e retornam ponteiro para nova instância do TAD), destrutores (libera a memória alocada dinamicamente), funções de acesso (observadoras – retornam o valor de um parâmetro de interesse mas não modificam o TAD) e função de manipulação do TAD (recebem como parâmetro ponteiros para uma ou mais instâncias do TAD e, em geral, modificam apenas o primeiro argumento).

Um exemplo dessa última classe de funções é a função somar do TAD bignum, que recebe dois ponteiros para instâncias do bignum. A função executa a operação de soma e armazena o resultado no TAD apontado pelo primeiro ponteiro enquanto que o TAD apontado pelo segundo ponteiro não é alterado. Além disso, buscando encapsular os dados dos TAD's, a definição do struct correspondente foi inserida no arquivo fonte, no caso dos TAD's digit e list e um nome de referência criado para uso geral no arquivo cabeçalho.

No caso do TAD bignum, essa meta não foi atendida devido à escassez de tempo e planejamento inadequado da sua forma de interação / interface ao utilizador. Assim, dessa forma, a definição do TAD bignum aparece no arquivo cabeçalho para permitir o uso de recursos, como o acesso direto à lista encadeada através do operador seta.

O TAD digit é o único independente dos demais, por ser o TAD mais básico desse projeto. Incluem-se nesse TAD duas bibliotecas padrões da linguagem C (stdlib.h e stdio.h) para permitir a exibição de mensagens de erro, depuração do código do TAD e outras funções.

O TAD list é construído usando a interface oferecida pelo TAD digit. Para atender aos requisitos de operação do TAD bignum, forneceu-se duas funções inserção

apenas: uma para inserção de dígito na frente do número e outra para inserção ao final. A fim de apagar qualquer dígito da lista encadeada, forneceu-se uma função para receber o ponteiro da posição a ser apagada.

O TAD bignum, por sua vez, estava programado para ter interface enxuta, mas com o decorrer de sua elaboração foi sendo expandido para acomodar as necessidades das funções obrigatórias. Por exemplo, não implementou-se uma função específica para verificar condições de igualdade, maior ou menor em um única função. No entanto, ao final, foram desenvolvidas funções diferentes para o fornecimento dessa informação. Grande parte das manipulações do TAD bignum foi executada usando as ferramentas do TAD list e do TAD digit diretamente, através do ponteiro list_num. Esse TAD, portanto, depende do TAD list e do TAD digit.

Tabela 1: Listagem das funções da interface de cada TAD desse trabalho

TAD digit	TAD list	TAD bignum
DIGIT* create_digit()	LIST* create_list ()	BIGNUM* criar()
void destruir_digit (DIGIT** f)	void destruir_list(LIST** l)	void destruir(BIGNUM** b)
void set_dig (DIGIT* d, char new_d)	DIGIT* get_first (LIST* l)	void ler_tela(BIGNUM* b)
void set_previous (DIGIT* d, DIGIT* new_previous)	DIGIT* get_last (LIST* l)	void ler_texto(FILE* infile, BIGNUM* b)
void set_next (DIGIT* d, DIGIT* new_next)	unsigned long int get_size (LIST* l)	void escrever_texto(FILE* outfile, BIGNUM* b)
char get_dig (DIGIT* d)	void set_first(LIST* l, DIGIT* new_first)	void imprimir(BIGNUM* A, char sci)
DIGIT* get_previous(DIGIT* d)	void set_last(LIST* l, DIGIT* new_last)	void copiar(BIGNUM* A, BIGNUM* B)
DIGIT* get_next(DIGIT* d)	void set_size(LIST* l, unsigned long int new_size)	void somar(BIGNUM* A, BIGNUM* B)
	void insert_begin(LIST* l, char d)	void subtrair(BIGNUM* A, BIGNUM* B)
	void insert_end(LIST* l, char d)	void multiplicar(BIGNUM* A, BIGNUM* B)
	void erase_digit(LIST* l, DIGIT** er)	void dividir(BIGNUM* A, BIGNUM* B)
	void clear_list(LIST* l)	void negativo (BIGNUM* A)
	void copyAtoB(LIST* A, LIST* B)	int maior_modulo(BIGNUM* A, BIGNUM* B)
	void print_list(LIST* l)	void long_to_big(BIGNUM* A, unsigned long int b)
		int igual (BIGNUM* A, BIGNUM* B);
		int nulo(BIGNUM* A);
		void fatorial(BIGNUM* A);
		void combinacao(BIGNUM* A, BIGNUM* B);

5. FORMATO DA ENTRADA E SAÍDA DOS DADOS

O programa principal, função main, recebe como argumento o nome do arquivo texto onde se encontram um conjunto de duplas de números inteiros, possivelmente

números de precisão arbitrária. Cada linha do arquivo contém apenas uma dupla. O programa inicia a leitura das duplas na primeira linha, 1, e calcula a partir da dupla (a, b) a combinação (a,b). A leitura e cálculo das combinações prossegue até ser encontrada uma dupla de zeros, indicando o final do arquivo.

O resultado das combinações das linhas ímpares é somado à um número de precisão arbitrária R, com a função de acumular os resultados. O resultado correspondente das linhas pares é subtraído da variável R. Ao final da leitura do arquivo, esse resultado é armazenado em um arquivo de saída, com o mesmo nome do arquivo de entrada e com extensão .out.

Exemplo de entrada de dados:

100	6	
120	110	
123456	123455	
12	11	
1	0	
133	28	
336	97	
401	35	
422	15	
438	247	
87	60	
19	16	
198	2	
12.345.678.901.234.567.890		12.345.678.901.234.567.890
98.765.432.109.876.543.210		98.765.432.109.876.543.209
0	0	

Saída correspondente:

```
752.780.542.029.917.574.986.472.383.546.393.218.499.515.705.91
1.109.187.664.196.638.566.881.275.494.391.695.150.757.570.578.
702.136.901.379.409.357.806.462.111.940.369.036
```

O tempo gasto para processamento desse arquivo de dados foi de cerca de 12 segundos. Por limitações de tempo, não foram executados testes extensivos de desempenho do programa como um todo.

6. REFERÊNCIAS BIBLIOGRÁFICAS

1. **Arbitrary-precision arithmetic.** Disponível em:
<http://en.wikipedia.org/wiki/Arbitrary-precision_arithmetic>. Acesso em: 28 ago. 2011.
2. **Arbitrary Precision Arithmetic.** Disponível em:
<<http://www.cs.sunysb.edu/~algorithm/files/high-precision-arithmetic.shtml>>. Acesso em 28 ago. 2011.

7. APÊNDICE – ARQUIVO MAKEFILE

Segue abaixo transcrição do arquivo Makefile desse trabalho:

```
all:tp0

tp0: main.o digit.o list.o
    gcc -o tp0 main.o digit.o list.o

main.o: main.c
    gcc -Wall -c -o main.o main.c

digit.o: digit.c
    gcc -Wall -c -o digit.o digit.c

list.o: list.c
    gcc -Wall -c -o list.o list.c

list.o: digit.h
```